

4. Binary Search with Big-O Analysis

Aim: To search for an element in a sorted array using Binary Search and analyze its time complexity.

Algorithm

1. Read a sorted array.
2. Set $low = 0$ and $high = n - 1$.
3. Find the middle element.
4. If the middle element equals the search element, return its position.
5. If the search element is smaller, search the left half.
6. Otherwise, search the right half.
7. Repeat until the element is found or the search range becomes empty.

Python Program

```
arr = list(map(int, input("Enter sorted array: ").split()))
```

```
key = int(input("Enter element to search: "))
```

```
low = 0
```

```
high = len(arr) - 1
```

```
found = False
```

```
while low <= high:
```

```
    mid = (low + high) // 2
```

```
    if arr[mid] == key:
```

```
        print("Element found at position:", mid + 1)
```

```
        found = True
```

```
        break
```

```
    elif key < arr[mid]:
```

```
        high = mid - 1
```

```
    else:
```

```
        low = mid + 1
```

```
if not found:
```

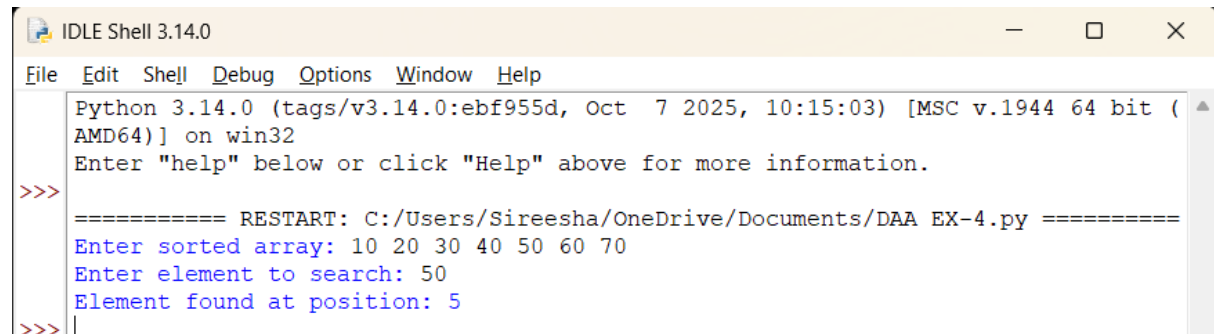
```
    print("Element not found")
```

Input

Enter sorted array: 10 20 30 40 50 60 70

Enter element to search: 50

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/DAA EX-4.py =====
Enter sorted array: 10 20 30 40 50 60 70
Enter element to search: 50
Element found at position: 5
>>>
```

Result

Thus, the Python program to search for an element using Binary Search was successfully executed, and its Big-O time complexity was analyzed.